AEGISLEDGER v1.0

Enterprise Core Banking & Immutable Double-Entry Ledger System

Software Requirements Specification (SRS)

Document Version	1.0.0 (Production Release Candidate)
Target Platform	.NET 10 (C# 14) / Next.js 15 / React Native 0.76
Database & Caching	SQL Server 2022 (Columnstore & RowVersion) + Redis 7
System Invariant	SUM(Debits) == SUM(Credits) [100% Zero-Drift Verified]
Primary Author	Senior .NET Lead Engineer & Solutions Architect
Security Target	ACID Isolation: SERIALIZABLE | OAuth2 / JWT / RBAC

Table of Contents

Table of Contents 2
1. Executive Summary & System Vision 3
2. System Architecture & Technical Stack 3
2.1 Solution Layering (.NET 10 Clean Architecture) 3
2.2 Technology Stack Summary 4
3. Functional Requirements Specification 4
3.1 FR-1: Accounts & Chart of Accounts (COA) Management 4
3.2 FR-2: Immutable Double-Entry Journal & Posting Pipeline 5
3.3 FR-3: Transaction Holds, Ring-Fencing & Escrow Engine 5
3.4 FR-4: Asynchronous EOD Reconciliation & Automated Hangfire Workers 5
3.5 FR-5: Identity, Security, RBAC & Multi-Currency Switcher 6
4. Client Interface Specifications 6
4.1 Next.js 15 Web Admin Portal (Operations & Compliance) 6
4.2 React Native Mobile Wallet Client (Consumer App) 6
5. Database Schema & Data Architecture (SQL Server) 7
5.1 Core Tables Definition 7
6. API Contract & Endpoints Map (.NET 10 Minimal APIs) 7
7. Non-Functional Requirements (NFRs) 8
8. Implementation Roadmap & Deployment Plan 8

1. Executive Summary & System Vision

AegisLedger v1.0 represents a financial-grade Core Banking and Double-Entry General Ledger System designed to enforce immutable transaction integrity, multi-account solvency, and zero-drift balance consistency across commercial banking operations. Modern fintech and core banking backends demand mathematical certainty: money cannot be created or destroyed, available liquidity must be strictly separated from book balances, and concurrent ledger transactions must execute with strict serializability.

The primary objective of AegisLedger is to demonstrate top-tier .NET enterprise architecture capable of powering retail deposits, commercial checking, multi-party escrow, and inter-bank clearance. By combining ASP.NET Core (.NET 10), Clean Architecture, CQRS via MediatR, EF Core Code-First with Dapper read-side optimization, SQL Server optimistic RowVersion concurrency locks, and distributed Redis idempotency protection, AegisLedger guarantees that every transaction is posted cleanly with net-zero invariant drift: **SUM(Debits) == SUM(Credits).**

> **CORE BANKING SOLVENCY INVARIANT**
> System Solvency Rule: The system ledger enforces continuous mathematical solvency where Total Book Ledger Balance ($12,450,890.00) equals Total Available Liquidity ($12,110,230.00) plus Active Security Holds ($340,660.00). Any drift between total debits and total credits immediately triggers a global circuit breaker.

2. System Architecture & Technical Stack

AegisLedger follows a strict Clean Architecture pattern decomposed into four decoupled projects within a Modular Monolith solution structure, flanked by an enterprise Next.js Web Admin Portal and a React Native Mobile Wallet.

2.1 Solution Layering (.NET 10 Clean Architecture)

1. AegisLedger.Domain: Contains domain entities (Accounts, Transactions, LedgerEntries, TransactionHolds), Enums (AccountType, EntryType, HoldReason, TransactionStatus), Value Objects (Money, Currency), and domain invariants. Zero external dependencies.

2. AegisLedger.Application: Implements CQRS commands and queries via MediatR. Features FluentValidation pipeline behaviors, custom Exception Handling decorators, DTOs, and interface contracts for data access, Redis caching, and Hangfire background jobs.

3. AegisLedger.Infrastructure: Houses EF Core DbContext with SQL Server configuration, RowVersion optimistic locking, Dapper raw SQL query handlers for high-speed reporting, Redis cache management, and Hangfire job registration.

4. AegisLedger.Api: ASP.NET Core Minimal APIs & Controllers, JWT Bearer Token Authentication, Custom Idempotency Middleware, Serilog structured request logging, and OpenAPI / Swagger documentation.

2.2 Technology Stack Summary

Layer / Domain	Technology Choice	Architectural Role & Justification
Backend Framework	.NET 10 (C# 14)	ASP.NET Core Minimal APIs + Web APIs for high-throughput, low-latency execution.
Data Access (Write)	EF Core Code-First	Change tracking, atomic unit-of-work transactions, and optimistic RowVersion locks.
Data Access (Read)	Dapper	Micro-ORM for ultra-fast, un-tracked SQL execution for balance feeds and EOD reports.
Database Engine	SQL Server 2022	Relational consistency, SERIALIZABLE transaction isolation, and RowVersion columns.
Caching & Idempotency	Redis 7	Distributed key-value store for API idempotency locks and JWT refresh token storage.
Async & Background	Hangfire v1.8.14	Persistent background job scheduler with SQL Server backing for EOD reconciliation.
Web Admin Frontend	Next.js 15 (React 19)	Enterprise operations portal with Shadcn UI, real-time charts, and audit views.
Mobile Client	React Native 0.76	Cross-platform mobile wallet for customer account views and idempotent P2P transfers.

3. Functional Requirements Specification

3.1 FR-1: Accounts & Chart of Accounts (COA) Management

The Chart of Accounts (COA) module serves as the central directory for customer deposit liabilities and system ledgers. Every account must track two distinct balances: **Book Ledger Balance** (total actual posted ledger funds) and **Available Liquidity** (book balance minus active security holds).

- Account Types: Checking (CHK), Savings (SAV), Escrow (ESC), and System Ledgers (SYS Revenue, Assets, Liabilities).
- RowVersion Tokens: Every account record includes a byte[] RowVersion column (e.g. 0x00000000000007D4) auto-incremented by SQL Server to detect concurrent balance modifications.
- Solvency Metrics: Web dashboard top-bar must display real-time Total Book Ledger Balance ($12,450,890.00), Available Liquidity ($12,110,230.00), and Active Security Holds ($340,660.00).
- Account Actions: Support for searching by account number or legal name, filtering by account type, viewing account-specific ledger journal entries, and locking/freezing accounts.

3.2 FR-2: Immutable Double-Entry Journal & Posting Pipeline

The Double-Entry Journal is an append-only ledger log where transactions generate balanced pairs of Debit (DR) and Credit (CR) posting entries. Direct modification or deletion of posted journal entries is strictly prohibited.

- Balanced Posting Invariant: A transaction is only committed if SUM(Debits) - SUM(Credits) == $0.00 across all splits.
- Atomic Transfer Execution: An internal transfer from Source (Debit) to Destination (Credit) creates one Transaction record and two matching LedgerEntry records inside an EF Core IDbContextTransaction with SERIALIZABLE isolation.
- Idempotency Guarantee: Accepts X-Idempotency-Key headers (e.g., idem-cf68a6a8-35e9-48e4-7e60-b9e227108a52). Duplicate request keys within a 24-hour window return the cached response from Redis without duplicate posting.
- Journal Reversal Mechanic: Erroneous transactions are corrected exclusively via compensating reversal entries that post equal and opposite DR/CR lines with reference to the original Transaction ID.

3.3 FR-3: Transaction Holds, Ring-Fencing & Escrow Engine

The Holds and Escrow Engine allows compliance officers and automated risk policies to ring-fence customer funds without mutating the underlying book ledger balance.

- Hold Enforcement: Deducts the specified hold amount from Available Liquidity while keeping Book Ledger Balance unchanged.
- Operational Reasons: Supports predefined regulatory hold reasons including 'Card Auth Pre-check', 'AML Fraud Review', 'Cheque Clearance', and 'Escrow Milestone Guarantee'.
- Automatic Expiry TTL: Configurable time-to-live windows (30 Mins, 2 Hours, 6 Hours, 24 Hours). Upon timer expiration, Hangfire workers execute background hold releases.
- Hold Lifecycle: States transition from Active -> Released (funds restored to Available) or Captured (funds debited permanently via journal transaction).

3.4 FR-4: Asynchronous EOD Reconciliation & Automated Hangfire Workers

AegisLedger incorporates automated End-of-Day (EOD) batch proof verification to guarantee that no historical accounting drift has occurred across the enterprise.

- Hangfire Scheduler: Executes the EODReconciliationJob daily at 00:00 UTC (or manually triggered via administrative button).
- Batch Verification Proof: Aggregates total system debits and total system credits, verifying a $0.00 difference across millions of historical entries.
- Cryptographic Audit Hashes: Generates an immutable SHA-256 hash (e.g., 0x8f2a991bce4718...) representing the entire state of the ledger for that audit date.
- Performance Logging: Logs batch execution duration (e.g., 420ms) and status in the EodReconciliationBatches table.

3.5 FR-5: Identity, Security, RBAC & Multi-Currency Switcher

- ASP.NET Core Identity & JWT: Secure user authentication with JWT access tokens (15-min TTL) and refresh tokens stored in Redis.
- Role-Based Access Control (RBAC): Enforces granular permissions for roles including 'Compliance Auditor', 'Bank Teller', and 'System Admin'.
- Multi-Currency Display: Frontends provide real-time currency conversion previews across USD, EUR, and PKR using cached central bank exchange rates.
- Correlation ID Tracking: All incoming HTTP requests and background job executions append a unique Correlation ID (e.g., corr-8f92a1b3) passed to Serilog.

4. Client Interface Specifications

4.1 Next.js 15 Web Admin Portal (Operations & Compliance)

Designed for high-density enterprise operations, featuring a dark slate theme (#0F172A), live status badges, and dynamic workspace tabs:

- **Dashboard Overview:** Real-time KPI metric cards (Total Deposits, Available vs Book, Active Holds, Today's Txn Throughput 1,420 Txns @ 99.98% success, P99 Latency 14ms), Live Double-Entry Flow Bar Chart, and 30-Day EOD Reconciliation Integrity chart.
- **Accounts & COA:** Data table displaying customer and system ledgers, balances, RowVersion hex tokens, status, and action buttons to view specific account journal entries.
- **Double-Entry Journal:** Filterable journal feed displaying reference IDs, timestamps, sender/receiver, amount, status, and expandable debit/credit journal split views.
- **Holds & Escrow Workspace:** Active holds table with countdown expiration timers, reason tags, and manual 'Release Hold' or 'Capture Hold' buttons.
- **EOD Audit & Reconcile:** Hangfire job status monitor, batch proof history table with SHA-256 hashes, and a 'Trigger Manual EOD Reconciliation' action button.

4.2 React Native Mobile Wallet Client (Consumer App)

Targeted at banking retail customers for secure, idempotent P2P money transfers and real-time account visibility:

- **Biometric Authentication:** FaceID / Fingerprint login integrating with ASP.NET Core Identity JWT token refresh flows.
- **Wallet Dashboard:** Displays Available Balance ($3,690,250.00) vs Book Ledger Balance ($3,840,250.00) with active hold notifications.
- **Initiate Transfer Screen:** Source/Destination account pickers, transfer amount inputs with quick % buttons (10%, 25%, 50%, MAX), transaction memo field, and client-side UUID idempotency key generator.

- **Transaction Split Preview:** Pre-commit modal showing simulated Debit (-$5,000.00) and Credit (+$5,000.00) lines with Net Diff: $0.00 [BALANCED] validation before user commitment.
- **Push Notifications:** Real-time alerts triggered by SignalR / FCM upon hold placement, hold expiry, or incoming funds credit.

5. Database Schema & Data Architecture (SQL Server)

The SQL Server database relational design enforces strict primary key GUIDs, foreign key relationships, index optimization, and RowVersion optimistic locking.

5.1 Core Tables Definition

Table Name	Primary Key	Key Columns & Data Types	Constraints & Indexes
Accounts	AccountId (GUID)	AccountNumber (NVARCHAR 20), LedgerBalance (DECIMAL 18,4), AvailableBalance (DECIMAL 18,4), RowVersion (ROWVERSION)	UNIQUE(AccountNumber), IX_Accounts_UserId, Concurrency Check on RowVersion
Transactions	TransactionId (GUID)	ReferenceNumber (NVARCHAR 50), IdempotencyKey (NVARCHAR 100), Amount (DECIMAL 18,4), Status (INT)	UNIQUE(ReferenceNumber), UNIQUE(IdempotencyKey), IX_Txn_CreatedAtUtc
LedgerEntries	EntryId (GUID)	TransactionId (FK), AccountId (FK), EntryType (1=DR, 2=CR), Amount (DECIMAL 18,4), PostBalance	FK_Ledger_Txn, FK_Ledger_Acc, Clustered Index on (AccountId, CreatedAtUtc)
TransactionHolds	HoldId (GUID)	AccountId (FK), HeldAmount (DECIMAL 18,4), Reason (NVARCHAR 100), Status, ExpiresAtUtc	FK_Holds_Acc, IX_Holds_Status_ExpiresAtUtc (For background Hangfire cleanup worker)
EodReconciliationBatches	BatchId (GUID)	AuditDate (DATE), TotalDebits (DECIMAL 18,4), TotalCredits (DECIMAL 18,4), Difference, BatchHash (NVARCHAR 64)	UNIQUE(AuditDate), IX_Eod_AuditDate DESC

6. API Contract & Endpoints Map (.NET 10 Minimal APIs)

All REST API endpoints adhere to JSON specifications, RFC 7807 ProblemDetails error standards, and JWT Bearer authorization.

HTTP Method	Endpoint Route	Required Headers / Body	Response & Status Code
POST	/api/v1/auth/token	Body: { username, password }	200 OK: { accessToken, refreshToken, expiresAt }

GET	/api/v1/accounts	Header: Authorization Bearer	200 OK: List<AccountDto> with RowVersion tokens
POST	/api/v1/transactions/transfer	Header: X-Idempotency-Key Body: { sourceAcc, destAcc, amount, memo }	201 Created: { txnId, refNum, status: 'Posted', netDiff: 0.00 }
POST	/api/v1/holds/apply	Body: { accountId, amount, reason, ttlMinutes }	200 OK: { holdId, newAvailableBalance, status: 'Active' }
POST	/api/v1/holds/{id}/release	Route: holdId GUID	200 OK: { holdId, status: 'Released', restoredBalance }
POST	/api/v1/reconciliation/eod	Role: Compliance Auditor	202 Accepted: { jobId, status: 'Scheduled in Hangfire' }

7. Non-Functional Requirements (NFRs)

- **Performance & Latency:** API P99 response time must remain below 15ms for transaction posting under standard loads (1,420 Txns throughput baseline). Dapper read queries must execute in <5ms.
- **Data Consistency & ACID Isolation:** Transaction processing must enforce SERIALIZABLE isolation level in SQL Server to eliminate phantom reads and double-spending race conditions.
- **Auditability & Traceability:** 100% of mutation operations must be captured in an append-only audit trail with request Correlation IDs, IP addresses, and user personas.
- **Resilience & Fault Tolerance:** System integration points utilize Polly resilience policies implementing exponential backoff retry (3 attempts) and circuit breaker mechanics.
- **Security Compliance:** Passwords hashed using Argon2id / Identity default PBKDF2; database connection strings and JWT signing keys stored in Azure Key Vault / Environment Variables.

8. Implementation Roadmap & Deployment Plan

Phase 1: Solution Setup & Domain Core

Initialize .NET 10 Solution structure, setup Domain Entities, Value Objects, DbContext, and EF Core SQL Server initial migrations with RowVersion.

Phase 2: Security & Idempotency Pipeline

Implement ASP.NET Core Identity, JWT authentication, Redis distributed caching, and custom Idempotency Middleware.

Phase 3: Double-Entry Engine & CQRS

Build MediatR Commands/Handlers for transaction transfers, FluentValidation pipeline, and xUnit concurrency unit tests.

Phase 4: Holds Engine & Dapper Optimization

Implement Holds/Escrow lifecycle, Dapper read queries for balance feeds, and Redis query caching.

🗓️ Phase 5: Hangfire Workers & Audit Logging

Configure Hangfire scheduler, build EODReconciliationJob with SHA-256 batch hash generation, and Serilog correlation tracking.

🗓️ Phase 6: Integration Testing & Error Resilience

Write integration tests using Microsoft.AspNetCore.Mvc.Testing, Polly circuit breakers, and RFC 7807 ProblemDetails middleware.

🗓️ Phase 7: Web Admin & Mobile App Connectors

Refine Next.js Web Portal and React Native Mobile Wallet to connect to ASP.NET Core REST API endpoints.

🗓️ Phase 8: Docker, Azure Deployment & Portfolio Demo

Configure docker-compose.yml (API, SQL Server, Redis, Hangfire), set up GitHub Actions CI/CD, deploy to Azure, and record public video walk-through.